

Sure — here is a clearer version you can send to an engineer. It explains **what we are building, how it should work, and what the final result must be.**

What are we building?

We need to build a **traffic video analysis system**.

The system receives a video from a **fixed CCTV camera looking at a road**. It must watch the video and automatically detect different traffic events, such as:

- accidents
- near misses
- red-light violations
- wrong-way driving
- illegal U-turns
- stopped vehicles
- jaywalking
- failure to yield to pedestrians
- illegal turns
- crossing solid lines
- stop-line violations
- congestion
- obstacles on the road
- fire/smoke

The camera does not move, so the road layout is always the same.

1. Understand the camera

We receive several sample videos.

First, we need to study them and understand:

- where the road is
- where each lane is
- traffic directions
- traffic lights

- pedestrian crossings
- stop lines
- road boundaries
- areas where vehicles normally move

Because the camera is fixed, we can manually define these areas once and reuse them.

2. Detect objects

We need an object-detection model such as:

YOLO / RT-DETR

The model should detect things like:

```
car
truck
bus
motorcycle
person
traffic light
```

We don't need to train an object detector from zero. We can use an existing open-source model and its weights.

3. Track objects

Detection only tells us:

"There is a car in this frame."

But we need to know:

"This is the same car that was here 3 seconds ago."

For that we use a tracker such as:

ByteTrack / DeepSORT

Example:

```
Frame 100:
Car #17 → (500, 300)
```

```
Frame 150:
Car #17 → (550, 320)
```

```
Frame 200:
Car #17 → (600, 350)
```

Now we have the car's trajectory.

4. Analyze movement

From the tracked objects we can calculate:

- position
- direction
- approximate speed
- acceleration/deceleration
- lane changes
- distance between vehicles
- whether a vehicle stopped
- whether two trajectories are approaching each other

This information is used to detect events.

5. Detect traffic events

We don't necessarily need an AI model for every event.

A lot of the logic can be **rule-based**.

For example:

Wrong way

```
Vehicle enters lane
      ↓
Movement direction is opposite
      ↓
wrong_way
```

Red light

```
Traffic light = RED
      ↓
Vehicle approaches stop line
      ↓
Vehicle crosses stop line
      ↓
red_light
```

Stopped vehicle

Vehicle stops
↓
Still stationary for ≥ 10 seconds
↓
stopped_vehicle

Jaywalking

Person
↓
Enters road
↓
Not inside pedestrian crossing
↓
jaywalking

Congestion

Many vehicles
↓
Very low movement
↓
All/most lanes affected
↓
congestion

Accident

Two objects approach each other
↓
Trajectories collide
↓
Contact / sudden stop
↓
accident

The exact rules need to be tuned using our sample videos.

6. The important function: `detect_events()`

The organizers will give our program an `.mp4` file.

Our function must return:

```
detect_events(video_path)
```

For example:

```
[
  [12.4, 18.9, "accident"],
  [40.0, 43.5, "red_light"],
  [72.1, 80.3, "jaywalking"]
]
```

Meaning:

```
12.4s → 18.9s    accident
40.0s → 43.5s    red_light
72.1s → 80.3s    jaywalking
```

The timing is important.

If the real accident happens from 20s to 25s, returning:

```
0s → 60s accident
```

is bad because the temporal overlap will be low.

So we need reasonably accurate **start and end times**.

Part B — Accident prediction

There is also a bonus part.

Instead of only detecting an accident **after it happens**, we should estimate the risk **before it happens**.

For every frame, we return a number:

```
0.0 = very low risk
1.0 = very high risk
```

Example:

```
10.0s → 0.02
10.5s → 0.03
11.0s → 0.08
11.5s → 0.25
12.0s → 0.65
12.5s → 0.88
13.0s → accident
```

The idea is:

```
Vehicles getting closer
      +
Sudden braking
      +
```

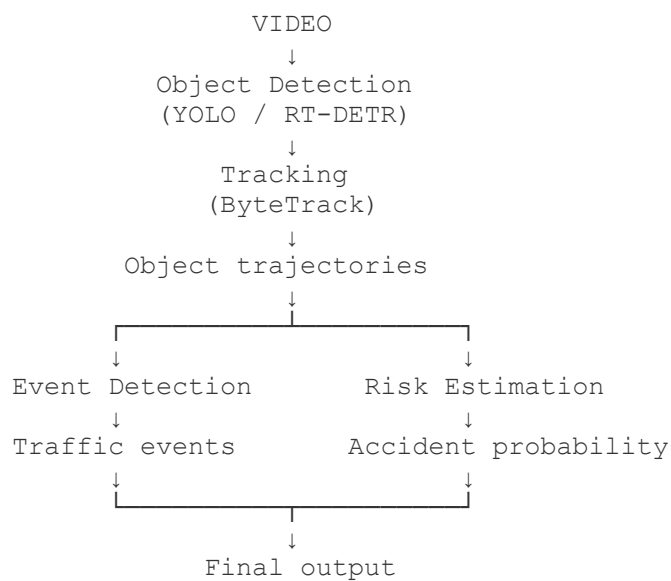
Dangerous trajectory
+
Wrong-way movement
↓
Higher accident risk

We have **5 seconds of anticipation**.

Importantly, the system is only allowed to use frames it has already seen. It cannot look into the future.

How the whole system works

The final pipeline should look roughly like this:



What we need to prepare

1. Development data

The organizers don't give us labels for the sample videos.

Therefore, we should manually annotate the sample videos ourselves:

```
start_time  
end_time  
event
```

For example:

```
15.2  20.8  red_light  
45.1  51.3  jaywalking
```

This becomes our **development/test set** for improving the rules.

2. Model

Use an open-source model.

Possible choices:

YOLO
RT-DETR

For tracking:

ByteTrack
DeepSORT

Everything must work **offline** during the competition.

The model weights must be included/downloaded before evaluation.

3. solution.py

The organizers expect exactly this interface:

```
def detect_events(video_path: str):  
    ...
```

and:

```
class RiskEstimator:  
  
    def reset(self, meta):  
        ...  
  
    def step(self, frame, t_sec):  
        ...
```

The organizers' program will call these functions automatically.

Website

We also need a public website.

It should show:

Team

Who we are and what each person did.

Approach

Show the pipeline:

Video
↓
YOLO
↓
ByteTrack
↓
Trajectory
↓
Rules
↓
Events

EDA

Show information about the sample videos:

- resolution
- FPS
- duration
- number of vehicles
- number of pedestrians
- traffic density
- trajectories
- movement directions
- heatmaps

Results

Show our processed sample videos.

For example:

00:15 ————— 00:21
RED LIGHT

And ideally show the detected objects directly on the video.

Live demo

A visitor uploads an .mp4.

Our system processes it and displays:

Detected Events

00:12 - 00:18 red_light
00:31 - 00:35 jaywalking
00:52 - 00:58 wrong_way

It should also visualize the events on a timeline/video.

Final repository

Our GitHub repository should roughly look like:

```
project/
├── solution.py
├── run_submission.py
├── evaluate.py
├── requirements.txt
├── weights/
│   └── model.pt
├── src/
│   ├── detector.py
│   ├── tracker.py
│   ├── trajectories.py
│   ├── events.py
│   ├── risk.py
│   └── rules.py
├── notebooks/
│   └── EDA.ipynb
├── predictions_samples.json
└── README.md
```

The most important thing

We do not need to invent a completely new AI model.

The practical approach is:

```
Existing AI model
+
Object tracking
+
Our understanding of this specific camera
+
Rules for traffic events
+
Risk calculation
=
Complete solution
```

The biggest challenge is not simply detecting cars.

The challenge is **turning the movement of cars and people into the exact 14 event classes with accurate start/end times**, while keeping the program fast enough to process the hidden videos offline.